Correctness

Outline

Correctness of an algorithm
- Loop invariants

Correctness of Find Maximum

Correctness of Insertion Sort
- The inner loop
- The outer loop

Correctness of Binary Search

Correctness of the Euclidean algorithm

Correctness of recursive algorithms

Correctness of an algorithm

Introduction: Correctness of an algorithm

- In this section we define precisely when an algorithm is **correct**
- Before asking how fast an algorithm is, we must be sure that it does what it promises
- We introduce the standard tool to prove the correctness of a loop: the **loop invariant**, with its three properties (initialization, maintenance, termination)

Before speed, correctness

- The previous chapters measured how much an algorithm costs: the size of its input and its running time
- Before asking how fast an algorithm is, however, we must be sure that it does what it promises
- This chapter presents the standard tool to prove it, the **loop invariant**, and applies it to algorithms already met in the chapters «Sorting», «Linear and binary search» and «Numerical algorithms»:
  - Find Maximum, Insertion Sort, Binary Search and the Euclidean algorithm.

Correctness must be proved

- The first part of the analysis of an algorithm consists in determining its **correctness**, i.e., proving that it computes the correct output for every possible input instance
- Testing an algorithm on some instances can show that it is **wrong** (one counterexample is enough)
- But it can never show that it is correct: there are infinitely many input instances
- Correctness must be **proved**

Definition 1: correct algorithm

*An algorithm is **correct** if, for every input instance I of the problem, it **terminates** after a finite number of steps and it **returns the correct output** for I.*

Correctness of loops

- The algorithms we consider consist of sequences of instructions and of loops
- Sequences of instructions are easy to analyse, while a loop executes the same instructions a number of times that depends on the input

The correctness of an algorithm is based on the correctness of its loops.

The correctness of a loop is proved by means of a **loop invariant**.

Definition 2: loop invariant

*A **loop invariant** is a property of the variables of an algorithm that is true every time the condition of a loop is evaluated, i.e., before the first iteration of the loop and after each iteration of the loop.*

The three properties to prove

To prove the correctness of an algorithm by means of a loop invariant, we must show three things:

1. **Initialization**: the loop invariant is true before the first iteration of the loop.
2. **Maintenance**: if the loop invariant is true before an iteration of the loop, then it remains true at the end of the iteration, or equivalently, it is true before the next iteration.
3. **Termination**: the loop terminates after a finite number of iterations. When the loop terminates, the loop invariant, together with the fact that the condition of the loop is false, gives a useful property that shows that the algorithm is correct.

Similarity with mathematical induction

- Note the similarity with **mathematical induction**, where to prove that a property holds it is necessary to prove a base case and an inductive step:
  - showing that the invariant holds before the first iteration corresponds to the **base case**;
  - showing that the invariant holds from iteration to iteration corresponds to the **inductive step**.

The termination property differs from the usual use of mathematical induction,
in which the inductive step is applied infinitely many times:
here, we stop the "induction" when the loop terminates.

How to prove that a loop terminates

- To prove that a loop terminates, we usually identify a **natural number**, for example

$$n - i + 1 \quad \text{or the length of a sub-array,}$$

that:

  - strictly decreases at every iteration;
  - must be positive for the loop to continue.

- Since a natural number cannot decrease forever, the loop terminates

Scheme of a correctness proof with a loop invariant

1. State the invariant: a property of the variables, true every time the condition of the loop is evaluated.
2. **Initialization**: check it with the values assigned before the loop.
3. **Maintenance**: assume it and the condition of the loop, execute the body once, and check it with the new values.
4. **Termination**: find a natural number that strictly decreases at each iteration; then combine the invariant with the negated condition of the loop to obtain the correct output.

Summary

Correct algorithm:

- for every input instance, it terminates and returns the correct output
- testing can only show that an algorithm is wrong: correctness must be proved

Loop invariant:

- a property of the variables, true every time the condition of the loop is evaluated
- three properties: initialization, maintenance, termination
- initialization and maintenance are the base case and the inductive step of an induction that stops when the loop terminates

Correctness of Find Maximum

Introduction: Correctness of Find Maximum

- In this section we apply the loop invariant to the simplest loop: a single scan of an array
- We state the invariant of the `FindMaximum` algorithm, check it on an example, and prove the three properties
- The proof is the model for all the proofs of the chapter

The problem

Given an array $a[1..n]$ of n integer numbers, with $n \geq 1$,
consider the problem of determining
the maximum value in the array.

- The first example is the simplest loop: a single scan of the array
- This is the pseudo-code of the `FindMaximum` algorithm

Algorithm: FindMaximum

Input : An array $a[1..n]$ of n integer numbers
Output : The maximum value in array a

```
1  function FindMaximum(a, n)
2      max := a[1]
3      i := 2
4      while i ≤ n do
5          if a[i] > max then
6              max ← a[i]
7          end
8          i ← i + 1
9      end
10     return max
11 end
```

Run in Colab: Python · C++ · Java

Loop invariant of FindMaximum

- The **loop invariant** of the while loop of `FindMaximum` is:

Every time the condition at line 4 is evaluated,
the variable *max* is equal to the maximum value of the sub-array $a[1..i-1]$:

$$max = \max\{a[1], a[2], \ldots, a[i-1]\}.$$

Example

Consider the array $a = [9, 12, 3, 16, 19, 11, 12]$ with $n = 7$. The table shows the values of i and max every time the condition at line 4 is evaluated:

i	max	$a[1..i-1]$	$\max\{a[1], \ldots, a[i-1]\}$	
2	9	$[9]$	9	
3	12	$[9, 12]$	12	
4	12	$[9, 12, 3]$	12	
5	16	$[9, 12, 3, 16]$	16	
6	19	$[9, 12, 3, 16, 19]$	19	
7	19	$[9, 12, 3, 16, 19, 11]$	19	
8	19	$[9, 12, 3, 16, 19, 11, 12]$	19	(loop ends)

At every evaluation the invariant holds. When the loop ends $i = n + 1 = 8$, and the algorithm returns 19, the maximum of the whole array.

Using the loop invariant we can prove the correctness
of the `FindMaximum` algorithm,
which is stated by the following observation.

Observation 1

Given an array $a[1..n]$ *of* $n \geq 1$ *integer numbers, the* `FindMaximum` *algorithm terminates and returns*

$$\max\{a[1], a[2], \ldots, a[n]\}.$$

Proof (1/3)

We prove the three properties of the loop invariant.

Initialization.

- Before the first iteration, lines 2 and 3 set $max = a[1]$ and $i = 2$
- The sub-array $a[1..i-1] = a[1..1]$ consists of the single element $a[1]$, whose maximum is $a[1] = max$
- Hence the invariant holds

Proof (2/3)

Maintenance. Assume that, before an iteration, $max = \max\{a[1], \ldots, a[i-1]\}$. The body of the loop compares $a[i]$ with max (line 5):

- if $a[i] > max$, then line 6 sets $max \leftarrow a[i]$, and $a[i]$ is greater than all the elements of $a[1..i-1]$;
- if $a[i] \leq max$, then max is not changed and it is greater than or equal to $a[i]$.

In both cases, at the end of the body

$$max = \max\{a[1], \ldots, a[i-1], a[i]\}.$$

Line 8 increases i by 1, so that, with the new value of i, we have again $max = \max\{a[1], \ldots, a[i-1]\}$. Hence the invariant holds before the next iteration.

Proof (3/3)

Termination.

- The variable i starts from $2 \leq n+1$ and increases by 1 at every iteration
- Therefore the loop terminates after exactly $n-1$ iterations, when for the first time $i = n+1$
- Substituting $i = n+1$ in the loop invariant we obtain

$$max = \max\{a[1], a[2], \ldots, a[n]\},$$

  which is the value returned at line 10

Hence, the algorithm is correct. □

Summary

Correctness of Find Maximum:

- invariant: $max = \max\{a[1], \ldots, a[i-1]\}$ every time the condition $i \leq n$ is evaluated
- initialization: $a[1..1]$ has the single element $a[1] = max$
- maintenance: comparing $a[i]$ with max extends the maximum to $a[1..i]$

Termination:

- exactly $n - 1$ iterations, the loop stops with $i = n + 1$
- with $i = n + 1$ the invariant gives the maximum of the whole array

Correctness of Insertion Sort

Introduction: Correctness of Insertion Sort

- In this section we prove the correctness of an algorithm with **two nested loops**
- The proof needs two invariants, one for each loop
- We first prove what a single execution of the inner loop does (a lemma), and then we use this result for the outer loop

The problem

Given an array $a[1..n]$ of n natural numbers, with $n \geq 1$, consider the problem of rearranging the elements of the array in non-decreasing order:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- With two nested loops the proof needs two invariants, one for each loop
- This is the pseudo-code of the `InsertionSort` algorithm

Algorithm: InsertionSort (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1 function InsertionSort(a, n)
2 |   i := 2
3 |   while i ≤ n do
4 |   |   key := a[i]                    // Element to insert
5 |   |   j := i − 1
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continued)
    while i ≤ n (continued) do
        while j ≥ 1 and a[j] > key do
            a[j + 1] ← a[j]  // Shift element to the right
            j ← j − 1
        end
        a[j + 1] ← key // Insert key in correct position
        i ← i + 1
    end
end
```

Run in Colab: Python · C++ · Java

The condition of the inner loop

- The condition at line 6 is evaluated from left to right, and the second part is evaluated only if the first one is true
- When $j = 0$ the condition is false and the non-existent element $a[0]$ is never accessed
- The C++ condition `i > -1 and a[i] > key` behaves in the same way

Plan of the proof

- The algorithm contains two nested loops
- We first prove what a single execution of the **inner** while loop (lines 6–8) does
- Then we use this result to prove the correctness of the **outer** while loop (lines 3–11)

The inner loop: notation

- Consider one iteration of the outer loop, with a given value of $i \in \{2, 3, \ldots, n\}$
- We denote by

$$b[1..i]$$

the content of the sub-array $a[1..i]$ at the beginning of this iteration (i.e., when the condition at line 3 is true)

- We assume that $b[1..i-1]$ is sorted in non-decreasing order
- After line 4 we have $key = b[i]$

Loop invariant of the inner loop

The **loop invariant** of the inner while loop is:

Every time the condition at line 6 is evaluated:

(I1) $a[1..j] = b[1..j]$ (the first j elements have not been moved);

(I2) $a[j+2..i] = b[j+1..i-1]$ (the elements $b[j+1], \ldots, b[i-1]$ have been shifted one position to the right);

(I3) every element of $a[j+2..i]$ is strictly greater than key.

The entries $a[i+1..n]$ are never modified by the inner loop.

The hole and the empty sub-arrays

- The position $j + 1$ is the "**hole**" in which *key* will be inserted: its current content is a copy that will be overwritten
- A sub-array $a[l..r]$ with $l > r$ is **empty**
- Any property of all the elements of an empty sub-array is trivially true

Example

Consider the array $a = [5, 2, 4, 1, 3]$ at the beginning of the iteration $i = 5$ of the outer loop: $b[1..5] = [1, 2, 4, 5, 3]$ and $key = 3$. The table shows the array every time the condition at line 6 is evaluated.

j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[1..j]$	$a[j+2..5]$	condition
4	1	2	4	5	3	$[1, 2, 4, 5]$	$[\,]$	$a[4] = 5 > 3$: true
3	1	2	4	5	5	$[1, 2, 4]$	$[5]$	$a[3] = 4 > 3$: true
2	1	2	4	4	5	$[1, 2]$	$[4, 5]$	$a[2] = 2 \leq 3$: false

At every evaluation $a[1..j] = b[1..j]$, $a[j+2..5] = b[j+1..4]$ and all the elements of $a[j+2..5]$ are greater than 3. When the loop ends $j = 2$, and line 10 sets $a[3] \leftarrow 3$, giving $a[1..5] = [1, 2, 3, 4, 5]$.

Lemma 1: inner loop of Insertion Sort

Consider an iteration of the outer loop of `InsertionSort` *with* $i \in \{2, \ldots, n\}$*, and let* $b[1..i]$ *be the content of* $a[1..i]$ *at the beginning of the iteration. If* $b[1..i-1]$ *is sorted in non-decreasing order, then the inner while loop terminates and, after line 10, the sub-array* $a[1..i]$ *contains the elements* $b[1], \ldots, b[i]$ *sorted in non-decreasing order, while* $a[i+1..n]$ *is unchanged.*

Proof (1/5)

We prove the three properties of the loop invariant (I1)–(I3).

Initialization.

- Before the first evaluation of the condition, $j = i - 1$ and nothing has been modified
- Hence $a[1..i-1] = b[1..i-1]$, i.e. (I1) holds
- The sub-arrays $a[j+2..i] = a[i+1..i]$ and $b[j+1..i-1] = b[i..i-1]$ are both empty, so (I2) and (I3) hold trivially

Proof (2/5)

Maintenance.

- Assume that (I1)–(I3) hold and that the condition at line 6 is true, i.e., $j \geq 1$ and $a[j] > key$
- By (I1), $a[j] = b[j]$, so $b[j] > key$
- Line 7 sets $a[j+1] \leftarrow a[j] = b[j]$; therefore, by (I2),

$$a[j+1..i] = b[j..i-1],$$

  and all these elements are greater than key (by (I3) and $b[j] > key$)

- Positions $1, \ldots, j$ are not modified

Proof (3/5)

Maintenance (continued).

- Line 8 sets $j \leftarrow j - 1$
- Denoting by $j' = j - 1$ the new value, we have

$$a[1..j'] = b[1..j'],$$

$$a[j'+2..i] = a[j+1..i] = b[j..i-1] = b[j'+1..i-1],$$

and all the elements of $a[j'+2..i]$ are greater than *key*

- Hence (I1)–(I3) hold before the next evaluation of the condition

Proof (4/5)

Termination.

- The variable j starts from $i - 1$ and decreases by 1 at every iteration
- Since the loop stops at the latest when $j = 0$, the inner loop executes at most $i - 1$ iterations and terminates
- When it terminates, either $j = 0$ or $a[j] = b[j] \leq key$. By (I1)–(I3):
  - $a[1..j] = b[1..j]$ is sorted and, if $j \geq 1$, its largest element $b[j]$ is $\leq key$;
  - $a[j + 2..i] = b[j + 1..i - 1]$ is sorted and all its elements are $> key$.

Proof (5/5)

Termination (continued).

- Line 10 sets $a[j+1] \leftarrow key = b[i]$. Therefore

$$a[1..i] = \big[\, b[1], \ldots, b[j],\ key,\ b[j+1], \ldots, b[i-1] \,\big],$$

  which contains exactly the elements $b[1], \ldots, b[i]$

- It is sorted in non-decreasing order, because $b[1..j]$ is sorted, $b[j] \leq key < b[j+1]$, and $b[j+1..i-1]$ is sorted

- Positions $i+1, \ldots, n$ are never accessed

Loop invariant of the outer loop

- The **loop invariant** of the outer while loop is:

Every time the condition at line 3 is evaluated,
the sub-array $a[1..i-1]$ consists of the elements originally in $a[1..i-1]$,
but sorted in non-decreasing order,
and the sub-array $a[i..n]$ is equal to the original sub-array $a[i..n]$.

Example

Consider the array $a = [5, 2, 4, 1, 3]$ with $n = 5$. The table shows the array every time the condition at line 3 is evaluated; the sorted sub-array $a[1..i-1]$ is highlighted in blue.

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	original $a[1..i-1]$
2	5	2	4	1	3	$[5]$
3	2	5	4	1	3	$[5, 2]$
4	2	4	5	1	3	$[5, 2, 4]$
5	1	2	4	5	3	$[5, 2, 4, 1]$
6	1	2	3	4	5	$[5, 2, 4, 1, 3]$ (loop ends)

Using the loop invariant and the lemma on the inner loop
we can prove the correctness of the `InsertionSort` algorithm.

Observation 2

Given an array $a[1..n]$ *of* $n \geq 1$ *natural numbers, the* `InsertionSort` *algorithm terminates and, at the end, the array* $a[1..n]$ *contains the original elements rearranged in non-decreasing order:*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

Proof (1/3)

We prove the three properties of the loop invariant of the outer loop.

Initialization.

- Before the first iteration, $i = 2$ (line 2) and the array has not been modified
- The sub-array $a[1..i-1] = a[1..1]$ consists of the single original element $a[1]$, and it is trivially sorted
- The sub-array $a[2..n]$ is the original one
- Hence the invariant holds

Proof (2/3)

Maintenance.

- Assume that the invariant holds and that $i \leq n$
- Then $a[1..i-1]$ contains the original elements of $a[1..i-1]$ sorted, and $a[i]$ is the original element $a[i]$
- By Lemma 1, after line 10 the sub-array $a[1..i]$ contains these i elements, i.e., the original elements of $a[1..i]$, sorted in non-decreasing order
- and $a[i+1..n]$ is unchanged, hence still equal to the original one
- Line 11 sets $i \leftarrow i+1$, so the invariant holds with the new value of i

Proof (3/3)

Termination.

- The variable i starts from $2 \leq n+1$ and increases by 1 at every iteration, and each inner loop terminates (Lemma 1)
- Therefore the outer loop terminates after exactly $n-1$ iterations, when $i = n+1$ for the first time
- Substituting $i = n+1$ in the loop invariant, the sub-array $a[1..n]$, i.e., the entire array, consists of the original elements sorted in non-decreasing order

Hence, the algorithm is correct. □

Summary (1/2)

Inner loop:

- invariant (I1)–(I3): $a[1..j] = b[1..j]$, $a[j+2..i] = b[j+1..i-1]$, and every element of $a[j+2..i]$ is $> key$
- position $j+1$ is the "hole" in which key is inserted
- it executes at most $i-1$ iterations; then line 10 puts key between $b[j] \leq key$ and $b[j+1] > key$

The lemma:

- if $b[1..i-1]$ is sorted, after line 10 the sub-array $a[1..i]$ contains $b[1], \ldots, b[i]$ sorted, and $a[i+1..n]$ is unchanged

Summary (2/2)

Outer loop:

- invariant: $a[1..i-1]$ contains the original elements of $a[1..i-1]$ sorted, and $a[i..n]$ is the original one
- maintenance follows from the lemma on the inner loop
- exactly $n-1$ iterations; with $i = n+1$ the whole array is sorted

Nested loops:

- one invariant for each loop, the inner one proved first

Correctness of Binary Search

Introduction: Correctness of Binary Search

- In this section we prove the correctness of the iterative binary search
- In the previous two algorithms the loop always runs until its condition becomes false
- Binary search can also stop early, with a **return** inside the loop: the proof must take **both exits** into account

The problem

Given an array $a[1..n]$ of n natural numbers
sorted in non-decreasing order and a target natural number m,
consider the problem of determining
if m is one of the elements of the array.

- Binary search can stop early, with a **return** inside the loop, and the proof must take both exits into account
- This is the pseudo-code of the (iterative) `BinarySearch` algorithm

Algorithm: BinarySearch (1/2)

Input : An array $a[1..n]$ of n natural numbers sorted in non-decreasing order and a target value m

Output : Index of m in a, or -1 if not found

```
1 function BinarySearch(a, n, m)
2     left := 1                                   // Left boundary
3     right := n                                  // Right boundary
4     while left ≤ right do
5         mid := left + ⌊(right − left)/2⌋        // Middle index
6         if a[mid] = m then
7             return mid                          // Element found
8         end
```

Algorithm: BinarySearch (2/2)

```
function BinarySearch(a, n, m) (continued)
    while left ≤ right (continued) do
 9      if a[mid] < m then
10          left ← mid + 1          // Search in right half
11      end
12      else
13          right ← mid − 1         // Search in left half
14      end
15  end
16  return −1                       // Element not found
17 end
```

Run in Colab: Python · C++ · Java

Loop invariant of BinarySearch

The **loop invariant** of the while loop of `BinarySearch` is:

Every time the condition at line 4 is evaluated:

(I1) $1 \leq \mathit{left}$ and $\mathit{right} \leq n$;

(I2) if m is in a, then m is in $a[\mathit{left}..\mathit{right}]$, that is,

$$m \in \{a[1], \ldots, a[n]\} \implies m \in \{a[\mathit{left}], \ldots, a[\mathit{right}]\}.$$

The search interval

- In words: the algorithm never discards a portion of the array that may contain m
- The sub-array $a[left..right]$ is called the **search interval**
- It contains $right - left + 1$ elements, and it is empty if $left > right$

Example

Search for $m = 10$ in $a = [3, 9, 11, 12, 12, 16, 19]$ ($n = 7$). The table shows the search interval every time the condition at line 4 is evaluated.

left	*right*	$a[left..right]$	action
1	7	$[3, 9, 11, 12, 12, 16, 19]$	$mid = 4$, $a[4] = 12 > 10 \Rightarrow right = 3$
1	3	$[3, 9, 11]$	$mid = 2$, $a[2] = 9 < 10 \Rightarrow left = 3$
3	3	$[11]$	$mid = 3$, $a[3] = 11 > 10 \Rightarrow right = 2$
3	2	$[\,]$	$left > right \Rightarrow$ return -1

Since 10 is not in a, the invariant (I2) is trivially true; when the interval becomes empty the algorithm correctly returns -1.

Observation 3

Given an array $a[1..n]$ of n natural numbers sorted in non-decreasing order and a natural number m, the `BinarySearch` *algorithm terminates; it returns an index k such that $a[k] = m$ if m is in a, and it returns -1 otherwise.*

Proof (1/5)

We prove the three properties of the loop invariant.

Initialization.

- Lines 2 and 3 set $left = 1$ and $right = n$
- Then (I1) holds
- and $a[left..right] = a[1..n]$ is the whole array, so (I2) holds

Proof (2/5)

Maintenance.

- Assume that (I1)–(I2) hold and that $left \leq right$
- Line 5 computes $mid = left + \lfloor (right - left)/2 \rfloor$
- Since $0 \leq \lfloor (right - left)/2 \rfloor \leq right - left$, we have

$$1 \leq left \leq mid \leq right \leq n,$$

  so $a[mid]$ is a valid entry of the array

- Three cases are possible

Proof (3/5)

Maintenance (continued): the three cases.

- $a[mid] = m$: the algorithm returns mid at line 7, which is a correct output.
- $a[mid] < m$: since the array is sorted, for every $k \leq mid$ we have $a[k] \leq a[mid] < m$. Hence m is not in $a[left..mid]$, and if m is in $a[left..right]$ then it is in $a[mid + 1..right]$. Line 10 sets $left \leftarrow mid + 1 \geq 1$, so (I1) and (I2) still hold.
- $a[mid] > m$: since the array is sorted, for every $k \geq mid$ we have $a[k] \geq a[mid] > m$. Hence, if m is in $a[left..right]$, then it is in $a[left..mid - 1]$. Line 13 sets $right \leftarrow mid - 1 \leq n$, so (I1) and (I2) still hold.

Proof (4/5)

Termination.

- Consider the number of elements of the search interval, $right - left + 1$
- In the second case the new number is $right - mid \leq right - left$
- In the third case it is $mid - left \leq right - left$
- In both cases it decreases by at least 1
- Since the loop continues only while $right - left + 1 \geq 1$, the loop terminates after at most n iterations

Proof (5/5)

Termination (continued). The algorithm ends in one of two ways:

- it returns mid with $a[mid] = m$, which is correct;
- the condition $left \leq right$ is false, i.e., the search interval $a[left..right]$ is empty. Since an empty sub-array does not contain m, by (I2) m is not in a, and the algorithm correctly returns -1 at line 16.

Hence, the algorithm is correct. □

Summary

Invariant of Binary Search:

- (I1) $1 \leq \mathit{left}$ and $\mathit{right} \leq n$
- (I2) if m is in a, then it is in the search interval $a[\mathit{left}..\mathit{right}]$
- the algorithm never discards a portion of the array that may contain m

Two exits:

- the **return** inside the loop, with $a[\mathit{mid}] = m$
- the empty search interval: by (I2), m is not in a and -1 is correct
- the size $\mathit{right} - \mathit{left} + 1$ decreases by at least 1: at most n iterations

Correctness of the Euclidean algorithm

Introduction: Correctness of the Euclidean algorithm

- In this section we prove the correctness of the Euclidean algorithm for the greatest common divisor
- Here the invariant is not a property of a portion of an array, but an **equality between two numbers** that never changes
- Termination is proved with the natural number m, which strictly decreases

The problem

Given two natural numbers $n, m \geq 1$,
consider the problem of computing
their greatest common divisor $\gcd(n, m)$.

- In the last example the invariant is not a property of a portion of an array, but an equality between two numbers that never changes
- This is the pseudo-code of the Euclidean algorithm

Algorithm: EuclideanAlgorithm

Input : Two natural numbers $n, m \geq 1$
Output : $\gcd(n, m)$

```
1 function EuclideanAlgorithm(n, m)
2     while m ≠ 0 do
3         r := n mod m
4         n ← m
5         m ← r
6     end
7     return n                          // m = 0:  n is the GCD
8 end
```

Run in Colab: Python · C++ · Java

Two facts on the greatest common divisor

- The proof is based on the following property of the greatest common divisor, proved in the chapter «Numerical algorithms»:

$$\gcd(n, m) = \gcd(m,\ n \bmod m) \qquad \text{for every } n \geq 1,\ m \geq 1.$$

- We also use the convention

$$\gcd(n, 0) = n \qquad \text{for } n \geq 1 :$$

every natural number divides 0, so the common divisors of n and 0 are the divisors of n, and the greatest of them is n

Loop invariant of the Euclidean algorithm

- We denote by n_0 and m_0 the values of n and m given in input
- The **loop invariant** of the while loop is:

Every time the condition at line 2 is evaluated,
$n \geq 1$, $m \geq 0$, and

$$\gcd(n, m) = \gcd(n_0, m_0).$$

Example

Consider $n_0 = 252$ and $m_0 = 105$. The table shows the values of n and m every time the condition at line 2 is evaluated:

n	m	$\gcd(n, m)$
252	105	21
105	42	21
42	21	21
21	0	21 (loop ends)

The value $\gcd(n, m) = 21$ never changes, and when $m = 0$ the algorithm returns $n = 21 = \gcd(252, 105)$.

Observation 4

Given two natural numbers $n_0, m_0 \geq 1$, *the* `EuclideanAlgorithm` *terminates and returns* $\gcd(n_0, m_0)$.

Proof (1/3)

We prove the three properties of the loop invariant.

Initialization.

- Before the first iteration $n = n_0 \geq 1$ and $m = m_0 \geq 1$
- So trivially $\gcd(n, m) = \gcd(n_0, m_0)$

Proof (2/3)

Maintenance.

- Assume that the invariant holds and that $m \neq 0$, i.e., $m \geq 1$
- Line 3 computes $r = n \bmod m$, with $0 \leq r < m$
- Lines 4–5 replace the pair (n, m) with (m, r)
- The new value of n is $m \geq 1$, the new value of m is $r \geq 0$, and, since $n, m \geq 1$,

$$\gcd(m, r) = \gcd(m,\ n \bmod m) = \gcd(n, m) = \gcd(n_0, m_0)$$

- Hence the invariant holds before the next iteration

Proof (3/3)

Termination.

- At every iteration the value of m is replaced by $r < m$, so m is a natural number that strictly decreases
- Therefore the loop terminates after at most m_0 iterations, when $m = 0$
- Substituting $m = 0$ in the loop invariant we obtain

$$\gcd(n_0, m_0) = \gcd(n, 0) = n,$$

  which is the value returned at line 7

Hence, the algorithm is correct. □

Two remarks

- The proof does not use the assumption $n_0 \geq m_0$: if $n_0 < m_0$, then n_0 mod $m_0 = n_0$ and the first iteration simply swaps the two numbers
- The bound "at most m_0 iterations" is enough to prove termination, but it is very pessimistic:
  - the number of iterations is actually $O(\log_2 m_0)$;
  - in the example, 3 iterations for $m_0 = 105$.

Summary

Invariant of the Euclidean algorithm:

- $n \geq 1$, $m \geq 0$ and $\gcd(n, m) = \gcd(n_0, m_0)$
- maintained by the property $\gcd(n, m) = \gcd(m,\ n \bmod m)$

Termination:

- m strictly decreases: at most m_0 iterations (actually $O(\log_2 m_0)$)
- with $m = 0$ the invariant gives $\gcd(n_0, m_0) = \gcd(n, 0) = n$

Correctness of recursive algorithms

Introduction: Correctness of recursive algorithms

- Loop invariants are the tool to prove the correctness of **iterative** algorithms, i.e., algorithms based on loops
- In this section we see how the correctness of a **recursive** algorithm is proved
- The tool is mathematical induction, applied directly on the size of the instance

Induction on the size of the instance

- A **recursive** algorithm solves an instance of a problem by calling itself on one or more smaller instances
- Its correctness is proved directly by **mathematical induction** on the size of the instance:
  - **base case**: the algorithm returns the correct output on the smallest instances, which are solved without recursive calls;
  - **inductive step**: under the inductive hypothesis that the recursive calls return the correct output on the smaller instances, the algorithm combines their results into the correct output for the current instance.

Termination of a recursive algorithm

- Termination follows from the fact that every recursive call is made on a strictly smaller instance
- So the base case is eventually reached

The correctness of recursive algorithms is proved by induction on the size of the input.

This technique is applied together with the study of recursive algorithms.

Summary

Iterative algorithms:

- correctness is proved with loop invariants

Recursive algorithms:

- correctness is proved by induction on the size of the instance
- base case: the smallest instances, solved without recursive calls
- inductive step: the recursive calls are assumed correct on the smaller instances
- termination: every recursive call is on a strictly smaller instance

Summary: key points of the chapter (1/2)

- An algorithm is correct if, for every input instance, it terminates and returns the correct output; testing can only show that an algorithm is wrong
- The correctness of a loop is proved with a loop invariant: initialization, maintenance and termination
- The invariant, together with the negated condition of the loop, gives the output:
  - $i = n + 1$ for Find Maximum and Insertion Sort;
  - an empty search interval for Binary Search;
  - $m = 0$ for the Euclidean algorithm.

Summary: key points of the chapter (2/2)

- Nested loops need one invariant for each loop (Insertion Sort)
- A **return** inside the loop is a second exit that must be proved correct (Binary Search)
- Recursive algorithms are proved correct by induction on the size of the input